

ASSIGNMENT 7.3

NAME:K SIRI CHANDANA

BATCH:27

HT NO: 2303A51937

Lab 7: Error Debugging with AI

Systematic approaches to finding and fixing bugs

Task 1: Fixing Syntax Errors

Buggy Code (missing colon):

Task 1: Fixing Syntax Errors

Buggy Code (missing colon):

Python ^

Copy

```
def add(a, b)
    return a + b
```

AI Correction:

Python ^

Copy

```
def add(a, b):
    return a + b
```

Explanation:

The error occurs because Python requires a colon (`:`) after the function definition line. Without it, the interpreter raises a `SyntaxError` . Adding the colon resolves the issue.

Task 2: Debugging Logic Errors in Loops

Task 1: Fixing Syntax Errors

Buggy Code (missing colon):

Python ^

Copy

```
def add(a, b)
    return a + b
```

AI Correction:

Python ^

Copy

```
def add(a, b):
    return a + b
```

Explanation:

The error occurs because Python requires a colon (`:`) after the function definition line. Without it, the interpreter raises a `SyntaxError`. Adding the colon resolves the issue.

Task 3: Handling Runtime Errors (Division by Zero)

Buggy Code:

Python ^

Copy

```
def divide(a, b):
    return a / b
```

AI Correction with Error Handling:

Python ^

Copy

```
def divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        return "Error: Division by zero is not allowed."
```

Explanation:

The program crashes when `b = 0`. Wrapping the division in a `try-except` block catches the `ZeroDivisionError` and prevents the program from terminating unexpectedly.

Task 4: Debugging Class Definition Errors

Buggy Code (missing `self`):

Python ^

 Copy

```
class Student:
    def __init__(name, age):
        name = name
        age = age
```

AI Correction:

Python ^

 Copy

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

Explanation:

In Python classes, `self` is required to refer to instance attributes. Without `self`, the variables are treated as local parameters and not stored in the object.

Task 5: Resolving Index Errors in Lists

Buggy Code (out-of-range index):

Python ^

 Copy

```
numbers = [1, 2, 3]
print(numbers[5])
```

AI Correction with Safe Access:

Python ^

 Copy

```
numbers = [1, 2, 3]
try:
    print(numbers[5])
except IndexError:
    print("Error: Index out of range. List length is", len(numbers))
```

Alternative Safe Access:

Python ^

 Copy

```
numbers = [1, 2, 3]
index = 5
if index < len(numbers):
    print(numbers[index])
else:
    print("Invalid index, maximum allowed:", len(numbers)-1)
```

Explanation:

Accessing `numbers[5]` causes an `IndexError` because the list only has 3 elements. Using bounds checking (`if index < len(numbers)`) or exception handling prevents the crash and provides a safe fallback.